ОТЧЁТ О ВЫПОЛНЕНИИ РАБОТЫ

# Веб-приложение на ASP.NET Core Razor Pages

Учёт грузов на складе: каталог, поиск, добавление, просмотр, редактирование и удаление. Публикация в Docker-контейнере на Render.

ASP.NET Core 9 | Razor Pages | Bootstrap 5 | Docker | Render

СТУДЕНТ
Юшкевич Владислав

ДИСЦИПЛИНА
Веб-технологии · лабораторная WT-5

ДАТА
26 октября 2026 г.

ТЕМА
ASP.NET Core Razor Pages, CRUD, Docker

РЕПОЗИТОРИЙ
github.com/Balur77/wt-lab5-5razor-yushkevich

ПРИЛОЖЕНИЕ
wt-lab5-5razor-yushkevich.onrender.com

Публичный URL: https://wt-lab5-5razor-yushkevich.onrender.com
Git-репозиторий: https://github.com/Balur77/wt-lab5-5razor-yushkevich

Отчёт составлен по фактически выполненной работе

Подпись: _____ Юшкевич В.

# Цель работы и задачи

Цель работы — изучить модель программирования **ASP.NET Core Razor Pages** и перевести учебное приложение учёта грузов с MVC на Razor Pages, а затем подготовить его к публикации в контейнере.

## Задачи

1. Разобрать структуру Razor Pages: страница `.cshtml` и её `PageModel`.
2. Реализовать обработчики `OnGet()` и `OnPost()`, привязку модели через `[BindProperty]`.
3. Реализовать полный CRUD: каталог, добавление, просмотр, редактирование и удаление груза.
4. Добавить поиск, фильтрацию, серверную и клиентскую валидацию, уведомления через `TempData`.
5. Оформить интерфейс на Bootstrap 5 с единым адаптивным layout.
6. Собрать Docker-образ, опубликовать приложение на Render, оформить Git-репозиторий и отчёт.

## Кратко о результате

| | |
|---|---|
| Тема | CargoFlow — учёт грузов на складе |
| Стек | .NET 9, ASP.NET Core Razor Pages, Bootstrap 5, jQuery Validation |
| Модель | `Cargo` + общее хранилище `CargoStore` в памяти |
| Страницы | Главная, каталог, добавление, подробности, редактирование, удаление |
| Контейнер | Docker, многоступенчатая сборка, порт из переменной `PORT` |
| Публикация | Render Web Service (Runtime: Docker), статус **Live** |
| Репозиторий | github.com/Balur77/wt-lab5-5razor-yushkevich |
| Приложение | wt-lab5-5razor-yushkevich.onrender.com |

> **ЧТО ПРОВЕРЕНО**
>
> Приложение собрано локально, запущено в Docker-контейнере и опубликовано на Render. Маршруты `/`, `/Cargo/Index`, `/Cargo/Create` и `/Cargo/Details?id=1` отвечают кодом **HTTP 200**.

# Домен и функциональность

CargoFlow автоматизирует базовый учёт грузов на складе. Основная сущность — **Cargo** (груз), перенесённая из лабораторной работы WT-4. Для каждой позиции хранятся название, категория, количество, место хранения, вес одной единицы, описание и признак доступности.

Общий вес партии и текстовый статус не дублируются в хранилище, а вычисляются в модели. Данные находятся в едином статическом хранилище `CargoStore`, поэтому добавленные записи видны на всех страницах до перезапуска приложения.

## Реализованные страницы

| Страница | Маршрут | Обработчики | Назначение |
|---|---|---|---|
| Главная | `/` | `OnGet()` | Показатели склада и последние добавленные грузы |
| Каталог | `/Cargo/Index` | `OnGet()` | Карточки грузов, поиск и фильтр по категории |
| Добавление | `/Cargo/Create` | `OnGet()`, `OnPost()` | Форма создания, валидация, сохранение, `TempData` |
| Подробности | `/Cargo/Details?id=` | `OnGet(int id)` | Полная карточка груза или `NotFound()` |
| Редактирование | `/Cargo/Edit?id=` | `OnGet(int id)`, `OnPost()` | Изменение полей существующей записи |
| Удаление | `/Cargo/Delete?id=` | `OnGet(int id)`, `OnPost(int id)` | Подтверждение и удаление груза |
| Ошибка | `/Error` | — | Стандартная страница обработки ошибок |

**6**
демонстрационных грузов в стартовом наборе

**5**
грузов доступно к отгрузке

**1 297**
единиц груза в каталоге

**1 547 кг**
суммарный вес всех партий

**ДОПОЛНИТЕЛЬНЫЕ ОПЕРАЦИИ**

Помимо обязательного добавления реализованы **просмотр**, **редактирование** и **удаление** — это закрывает полный CRUD-сценарий и дополнительные требования лабораторной работы.

# Структура проекта

Проект организован по стандартным соглашениям ASP.NET Core: страницы находятся в `Pages/`, общие фрагменты — в `Pages/Shared/`, модель и хранилище вынесены в отдельные папки.

```
wt-lab5-5razor-yushkevich/
├── Models/
│   └── Cargo.cs                # модель предметной области
├── Data/
│   └── CargoStore.cs           # единое хранилище в памяти
├── Pages/
│   ├── Index.cshtml(.cs)       # главная страница
│   ├── Cargo/
│   │   ├── Index.cshtml(.cs)   # каталог, поиск, фильтр
│   │   ├── Create.cshtml(.cs)  # добавление + валидация
│   │   ├── Details.cshtml(.cs) # подробная карточка
│   │   ├── Edit.cshtml(.cs)    # редактирование
│   │   └── Delete.cshtml(.cs)  # удаление
│   └── Shared/
│       ├── _Layout.cshtml      # navbar, footer, SVG-спрайт
│       ├── _CargoCard.cshtml   # переиспользуемая карточка
│       └── _ValidationScriptsPartial.cshtml
├── wwwroot/
│   ├── css/site.css            # тема и адаптивность
│   ├── js/site.js
│   ├── images/warehouse-hero.svg
│   └── lib/                    # Bootstrap, jQuery, валидация
├── Program.cs                  # культура, Razor Pages, PORT
├── Dockerfile
├── .dockerignore
├── README.md
└── wt-lab5-5razor-yushkevich.csproj
```

## Ключевые файлы

| Файл | Роль в приложении |
|---|---|
| `Program.cs` | Настройка Razor Pages, культуры `ru-RU` с точкой в десятичных числах, чтение `PORT` |
| `Models/Cargo.cs` | Модель с атрибутами валидации, вычисляемыми весом и статусом |
| `Data/CargoStore.cs` | Статический список грузов и выдача следующего `Id` |
| `Pages/Cargo/*.cshtml.cs` | PageModel: обработчики `OnGet` / `OnPost`, состояние страницы |
| `Pages/Cargo/*.cshtml` | Razor-разметка, Tag Helpers, формы и вывод данных |
| `Dockerfile` | Многоступенчатая сборка и запуск приложения в контейнере |

> **СОГЛАШЕНИЕ RAZOR PAGES**
>
> Страница и её PageModel лежат рядом и называются одинаково: `Index.cshtml` + `Index.cshtml.cs`. Класс наследуется от `PageModel`, а обработчики называются `OnGet` и `OnPost`.

# Модель Cargo

Модель описывает груз и одновременно задаёт правила валидации через DataAnnotations. Те же атрибуты `[Display]` используются Tag Helpers для подписей полей формы.

Models/Cargo.cs — Модель Cargo с атрибутами валидации

```cs
using System.ComponentModel.DataAnnotations;

namespace wt_lab5_5razor_yushkevich.Models;

/// <summary>
/// Груз на складе — модель предметной области, перенесённая из WT-4.
/// </summary>
public class Cargo
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Введите название груза")]
    [StringLength(100, ErrorMessage = "Название не должно превышать 100 символов")]
    [Display(Name = "Название")]
    public string Name { get; set; } = string.Empty;

    [Required(ErrorMessage = "Укажите категорию")]
    [StringLength(60, ErrorMessage = "Категория не должна превышать 60 символов")]
    [Display(Name = "Категория")]
    public string Category { get; set; } = string.Empty;

    [Range(1, 1_000_000, ErrorMessage = "Количество должно быть от 1 до 1 000 000")]
    [Display(Name = "Количество, шт.")]
    public int Quantity { get; set; } = 1;

    [Required(ErrorMessage = "Укажите место хранения")]
    [StringLength(100, ErrorMessage = "Место хранения не должно превышать 100 символов")]
    [Display(Name = "Место хранения")]
    public string Location { get; set; } = string.Empty;

    [Range(0.01, 1_000_000_000, ErrorMessage = "Вес должен быть положительным числом")]
    [Display(Name = "Вес, кг")]
    public decimal Weight { get; set; }

    [Required(ErrorMessage = "Укажите описание груза")]
    [StringLength(300, ErrorMessage = "Описание не должно превышать 300 символов")]
    [Display(Name = "Описание")]
    public string Description { get; set; } = string.Empty;

    [Display(Name = "Доступен на складе")]
    public bool IsAvailable { get; set; } = true;

    /// <summary>Общий вес партии.</summary>
    public decimal TotalWeight => Math.Round(Weight * Quantity, 2);

    /// <summary>Текстовое представление статуса для интерфейса.</summary>
    public string Status => IsAvailable ? "На складе" : "Нет в наличии";
}
```

Рис. 1. `Models/Cargo.cs` — модель, атрибуты валидации и вычисляемые свойства

## Что важно в модели

- ✓ `[Required]`, `[StringLength]` и `[Range]` задают серверные правила и автоматически попадают в клиентскую валидацию.
- ✓ `[Display(Name = ...)]` даёт русские подписи полей без дублирования текста в разметке.
- ✓ `TotalWeight` и `Status` вычисляются на лету — в хранилище хранятся только исходные данные.
- ✓ Модель не зависит от Razor Pages и может быть повторно использована в другом типе приложения.

# Единое хранилище CargoStore

В лабораторной работе не требуется база данных, поэтому используется статический список в памяти. Он общий для всех PageModel: страница добавления записывает объект, а каталог и главная сразу видят изменение.

Data/CargoStore.cs — Общее хранилище CargoStore (фрагмент)

```cs
using wt_lab5_5razor_yushkevich.Models;

namespace wt_lab5_5razor_yushkevich.Data;

/// <summary>
/// Единое хранилище грузов в памяти приложения.
/// Используется всеми Razor Pages, поэтому добавленные объекты
/// сохраняются при переходах между страницами.
/// </summary>
public static class CargoStore
{
    public static List<Cargo> Cargoes { get; } = new()
    {
        new Cargo
        {
            Id = 1,
            Name = "Ноутбуки Dell Latitude 5440",
            Category = "Электроника",
            Quantity = 24,
            Location = "Сектор А-1, стеллаж 3",
            Weight = 2.4m,
            Description = "Партия офисных ноутбуков для оснащения новых рабочих мест. Гарантия производителя — 24 месяца.",
            IsAvailable = true
        },
        new Cargo
        {
            Id = 2,
            Name = "Офисные кресла ErgoPro",
            Category = "Мебель",
            Quantity = 18,
            Location = "Сектор В-2, зона сборки",
            Weight = 12.5m,
            Description = "Эргономичные кресла с регулируемой поддержкой поясницы. Поставка в разобранном виде.",
            IsAvailable = true
        },
        new Cargo
        {
            Id = 3,
            Name = "Кофемашины Jura E8",
            Category = "Бытовая техника",
            Quantity = 6,
            Location = "Сектор С-1, стеллаж 1",
            Weight = 9.8m,
            Description = "Профессиональные кофемашины для переговорных комнат и клиентских зон.",
            IsAvailable = true
        },
```

Рис. 2. `Data/CargoStore.cs` — стартовый набор грузов и общий список

## Как устроено хранилище

- ✓ `static class CargoStore` содержит единственный `List<Cargo>` на всё приложение.
- ✓ При старте добавляются 6 реалистичных грузов разных категорий — интерфейс не пустой.
- ✓ `GetNextId()` возвращает максимальный `Id + 1`, поэтому новые записи не конфликтуют.
- ✓ После перезапуска приложения пользовательские записи исчезают — это осознанное ограничение учебного проекта.

**ОГРАНИЧЕНИЕ**

Хранилище не потокобезопасно и не сохраняется на диск. Для учебного сценария этого достаточно; в production здесь была бы база данных и репозиторий с внедрением зависимостей.

# PageModel каталога и OnGet()

`IndexModel` хранит параметры поиска и результаты выборки. Значения `Search` и `Category` связываются из query string, поэтому фильтр сохраняется в адресе страницы и работает без JavaScript.

Pages/Cargo/Index.cshtml.cs — PageModel каталога и обработчик OnGet()

```cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.AspNetCore.Mvc.Rendering;
using wt_lab5_5razor_yushkevich.Data;
using CargoModel = wt_lab5_5razor_yushkevich.Models.Cargo;

namespace wt_lab5_5razor_yushkevich.Pages.Cargo;

public class IndexModel : PageModel
{
    [BindProperty(SupportsGet = true)]
    public string? Search { get; set; }

    [BindProperty(SupportsGet = true)]
    public string? Category { get; set; }

    public IReadOnlyList<CargoModel> Cargoes { get; private set; } = Array.Empty<CargoModel>();
    public IEnumerable<SelectListItem> Categories { get; private set; } = Array.Empty<SelectListItem>();

    public int TotalCount => CargoStore.Cargoes.Count;
    public bool HasFilters => !string.IsNullOrWhiteSpace(Search) || !string.IsNullOrWhiteSpace(Category);

    public void OnGet()
    {
        var query = CargoStore.Cargoes.AsEnumerable();

        if (!string.IsNullOrWhiteSpace(Search))
        {
            var term = Search.Trim();
            query = query.Where(cargo =>
                cargo.Name.Contains(term, StringComparison.OrdinalIgnoreCase) ||
                cargo.Location.Contains(term, StringComparison.OrdinalIgnoreCase) ||
                cargo.Description.Contains(term, StringComparison.OrdinalIgnoreCase));
        }

        if (!string.IsNullOrWhiteSpace(Category))
        {
            query = query.Where(cargo =>
                string.Equals(cargo.Category, Category, StringComparison.OrdinalIgnoreCase));
        }

        Cargoes = query.OrderBy(cargo => cargo.Id).ToList();

        Categories = CargoStore.Cargoes
            .Select(cargo => cargo.Category)
            .Distinct(StringComparer.OrdinalIgnoreCase)
            .OrderBy(category => category)
            .Select(category => new SelectListItem(category, category))
            .ToList();
    }
}
```

Рис. 3. `Pages/Cargo/Index.cshtml.cs` — свойства страницы и обработчик `OnGet()`

## Логика выборки

- ✓ `Search` ищет подстроку без учёта регистра в названии, месте хранения и описании.
- ✓ `Category` фильтрует по точному совпадению категории.
- ✓ Результат сортируется по `Id`, а `Categories` формирует список для выпадающего меню.
- ✓ `TotalCount` и `HasFilters` используются в разметке для счётчика и бейджа активного фильтра.

# Razor Page каталога

Разметка получает данные из PageModel через `@Model`. Цикл `@foreach` выводит карточки, а `<partial>` подключает общий фрагмент `_CargoCard.cshtml` — разметка карточки не дублируется.

Pages/Cargo/Index.cshtml — Razor Page каталога: вывод карточек

```cshtml
<div class="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-3">
    <p class="text-muted mb-0">
        Найдено <strong>@Model.Cargoes.Count</strong> из <strong>@Model.TotalCount</strong> позиций
        @if (Model.HasFilters)
        {
            <span class="badge text-bg-light border ms-1">Фильтр активен</span>
        }
    </p>
    <span class="text-muted small">Данные хранятся в памяти приложения</span>
</div>

@if (Model.Cargoes.Count > 0)
{
    <div class="row g-4">
        @foreach (var cargo in Model.Cargoes)
        {
            <div class="col-12 col-md-6 col-lg-4">
                <partial name="_CargoCard" model="cargo" />
            </div>
        }
    </div>
}
else
{
    <div class="empty-state card border-0 shadow-sm rounded-4 text-center">
        <div class="card-body p-5">
            <span class="empty-state__icon">
                <svg class="icon" aria-hidden="true"><use href="#i-box" /></svg>
            </span>
            <h2 class="h4 fw-semibold mt-3">Ничего не найдено</h2>
            <p class="text-muted">
                Попробуйте изменить запрос или сбросить фильтры — в хранилище
                по-прежнему @Model.TotalCount позиций.
            </p>
            <a asp-page="/Cargo/Index" class="btn btn-primary">Сбросить фильтры</a>
        </div>
    </div>
}
```

**Рис. 4.** `Pages/Cargo/Index.cshtml` — счётчик, цикл по грузам и partial-карточка

## Использованные возможности Razor

- ✓ `@foreach` и `@if` — управляющие конструкции прямо в разметке.
- ✓ `<partial name="_CargoCard" model="cargo" />` — переиспользуемый компонент представления.
- ✓ `asp-page` и `asp-route-*` — генерация ссылок по именам страниц, а не по «сырым» URL.
- ✓ Bootstrap-классы `row g-4`, `col-12 col-md-6 col-lg-4` дают адаптивную сетку.

При пустом результате показывается отдельное состояние с предложением сбросить фильтры — страница не выглядит «сломанной».

# CreateModel: [BindProperty] и OnPost()

Форма добавления — центральный сценарий работы. Свойство `Cargo` помечено `[BindProperty]`, поэтому ASP.NET Core автоматически связывает поля POST-запроса с моделью до вызова `OnPost()`.

Pages/Cargo/Create.cshtml.cs — CreateModel: [BindProperty] и OnPost()

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using wt_lab5_5razor_yushkevich.Data;
using CargoModel = wt_lab5_5razor_yushkevich.Models.Cargo;

namespace wt_lab5_5razor_yushkevich.Pages.Cargo;

public class CreateModel : PageModel
{
    [BindProperty]
    public CargoModel Cargo { get; set; } = new();

    public IEnumerable<string> Categories => CargoStore.Cargoes
        .Select(cargo => cargo.Category)
        .Distinct(StringComparer.OrdinalIgnoreCase)
        .OrderBy(category => category);

    public void OnGet()
    {
    }

    public IActionResult OnPost()
    {
        if (!ModelState.IsValid)
        {
            return Page();
        }

        Cargo.Id = CargoStore.GetNextId();
        Cargo.Name = Cargo.Name.Trim();
        Cargo.Category = Cargo.Category.Trim();
        Cargo.Location = Cargo.Location.Trim();
        Cargo.Description = Cargo.Description.Trim();

        CargoStore.Cargoes.Add(Cargo);

        TempData["SuccessMessage"] =
            $"Груз «{Cargo.Name}» добавлен в каталог (ID: {Cargo.Id}).";

        return RedirectToPage("./Index");
    }
}
```

Рис. 5. Pages/Cargo/Create.cshtml.cs — привязка модели и обработчик POST

## Ключевые решения

- ✓ `ModelState.IsValid` — единая точка проверки серверной валидации; при ошибке возвращается та же страница.
- ✓ `Id` выдаётся хранилищем, а строковые поля очищаются от пробелов через `Trim()`.
- ✓ `TempData["SuccessMessage"]` переживает редирект и показывает одноразовое уведомление.
- ✓ `RedirectToPage("./Index")` реализует паттерн Post/Redirect/Get — форма не отправляется повторно при обновлении.

# Форма добавления груза

Форма собрана на Tag Helpers: они генерируют корректные `name` / `id`, подставляют значения, подписи и атрибуты клиентской валидации. Отдельный partial подключает jQuery Validation.

Pages/Cargo/Create.cshtml — Форма с Tag Helpers и валидацией

```
<div class="row g-3 g-md-4">
    <div class="col-12">
        <label asp-for="Cargo.Name" class="form-label fw-semibold">
            Название <span class="text-danger">*</span>
        </label>
        <input asp-for="Cargo.Name"
               class="form-control form-control-lg"
               placeholder="Например, ноутбуки Dell Latitude" />
        <span asp-validation-for="Cargo.Name" class="text-danger small"></span>
    </div>

    <div class="col-12 col-md-6">
        <label asp-for="Cargo.Category" class="form-label fw-semibold">
            Категория <span class="text-danger">*</span>
        </label>
        <input asp-for="Cargo.Category"
               class="form-control"
               list="category-options"
               placeholder="Электроника" />
        <datalist id="category-options">
            @foreach (var category in Model.Categories)
            {
                <option value="@category"></option>
            }
        </datalist>
        <span asp-validation-for="Cargo.Category" class="text-danger small"></span>
    </div>

    <div class="col-12 col-md-6">
        <label asp-for="Cargo.Location" class="form-label fw-semibold">
            Место хранения <span class="text-danger">*</span>
        </label>
        <input asp-for="Cargo.Location"
               class="form-control"
               placeholder="Сектор A-1, стеллаж 3" />
        <span asp-validation-for="Cargo.Location" class="text-danger small"></span>
    </div>
```

Рис. 6. `Pages/Cargo/Create.cshtml` — поля формы, `asp-for` и вывод ошибок

## Tag Helpers в форме

- ✓ `asp-for="Cargo.Name"` связывает input со свойством и добавляет `data-val-*` атрибуты.
- ✓ `asp-validation-for` выводит сообщение конкретного поля, `asp-validation-summary="ModelOnly"` — общие ошибки.
- ✓ `asp-items="Model.Categories"` заполняет выпадающий список категорий.
- ✓ `<partial name="_ValidationScriptsPartial" />` включает клиентскую проверку до отправки формы.

> **ДОСТУПНОСТЬ**
> У полей есть `label`, подсказки `placeholder`, обязательные поля отмечены звёздочкой, а кнопки имеют понятные тексты — форма удобна и с клавиатуры.

# Серверная и клиентская валидация

Один набор атрибутов модели работает с двух сторон: jQuery Validation проверяет форму в браузере, а `ModelState.IsValid` повторно проверяет данные на сервере. Сообщения локализованы на русский язык.

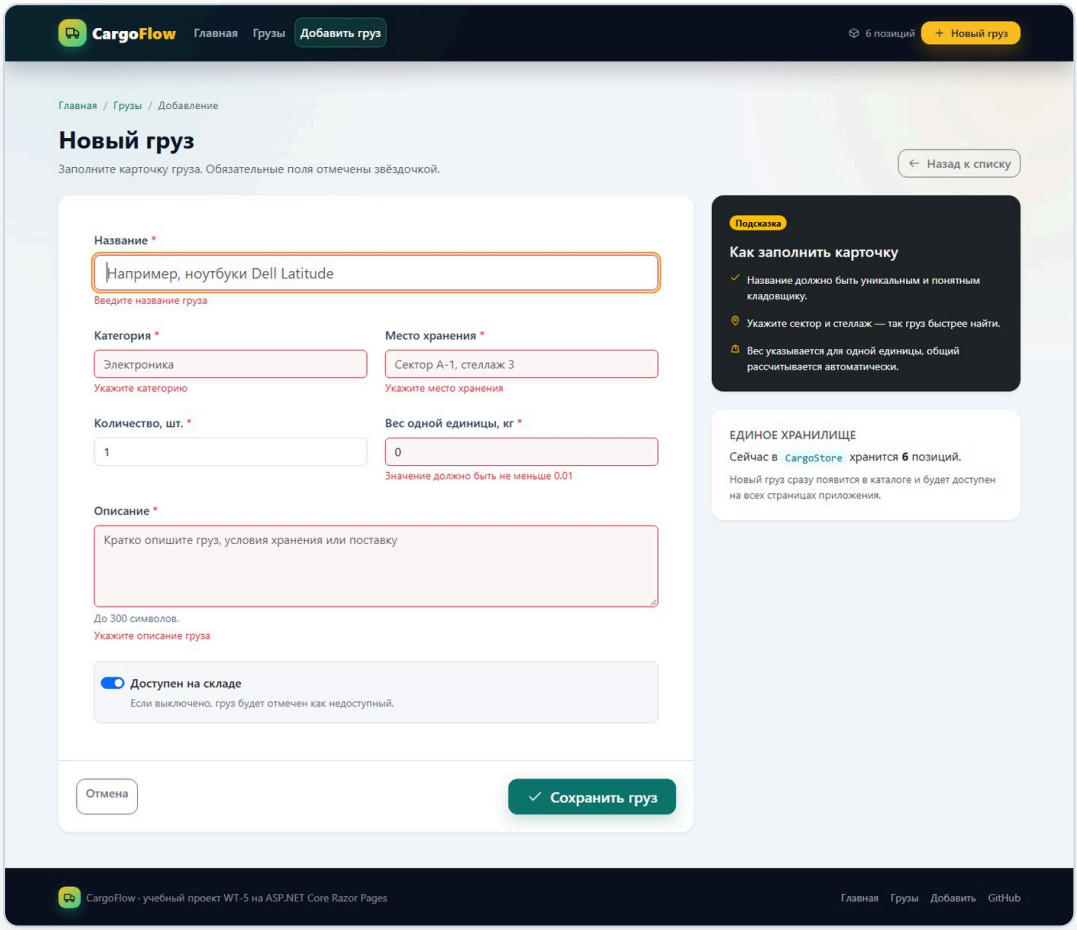



**Рис. 7.** Отправка пустой формы: клиентские сообщения об ошибках на русском языке

## Правила модели

| Поле | Ограничение |
|---|---|
| Название | обязательно, не более 100 символов |
| Категория | обязательно, не более 60 символов |
| Количество | целое число от 1 до 1 000 000 |
| Место хранения | обязательно, не более 100 символов |
| Вес единицы | от 0,01 до 1 000 000 000 кг |
| Описание | обязательно, не более 300 символов |

> **КУЛЬТУРА И ЧИСЛА**
>
> В `Program.cs` включена культура `ru-RU`, но десятичный разделитель принудительно оставлен точкой — иначе HTML `input type="number"` с дробным весом не связывался бы с моделью.

# Просмотр и удаление

Страница подробностей принимает `id` из query string и возвращает `NotFound()`, если запись не найдена. Удаление сделано в два шага: `OnGet` показывает подтверждение, `OnPost` удаляет запись.

**Details.cshtml.cs / Delete.cshtml.cs** — Дополнительные операции: Details и Delete

```
// Pages/Cargo/Details.cshtml.cs — OnGet(int id)
public class DetailsModel : PageModel
{
    public CargoModel? Cargo { get; set; }

    public IActionResult OnGet(int id)
    {
        Cargo = CargoStore.Cargoes.FirstOrDefault(cargo => cargo.Id == id);

        if (Cargo is null)
        {
            return NotFound();
        }

        return Page();
    }
}

// Pages/Cargo/Delete.cshtml.cs — OnPost(int id)
    public IActionResult OnPost(int id)
    {
        var cargo = CargoStore.Cargoes.FirstOrDefault(item => item.Id == id);

        if (cargo is null)
        {
            return NotFound();
        }

        CargoStore.Cargoes.Remove(cargo);

        TempData["SuccessMessage"] = $"Груз «{cargo.Name}» удалён из каталога.";

        return RedirectToPage("./Index");
    }
}
```

**Рис. 8.** Обработчики `Details.OnGet(int id)` и `Delete.OnPost(int id)`

## Почему удаление через POST

- ✓ GET-запрос не должен изменять состояние — иначе удаление сработает по простой ссылке или предзагрузке.
- ✓ Страница подтверждения показывает название груза, чтобы пользователь не удалил запись случайно.
- ✓ После удаления выполняется `RedirectToPage("./Index")` и показывается сообщение через `TempData`.
- ✓ Для отсутствующего `id` возвращается `NotFound()`, а не пустая страница.

# Редактирование груза

`EditModel` загружает запись по `id` в `OnGet`, а в `OnPost` проверяет модель и переносит изменённые поля в существующий объект хранилища. После сохранения — редирект в каталог с уведомлением.

Pages/Cargo/Edit.cshtml.cs — Редактирование: OnPost()

```
31      public IActionResult OnPost()
32      {
33          if (!ModelState.IsValid)
34          {
35              return Page();
36          }
37
38          var existing = CargoStore.Cargoes.FirstOrDefault(item => item.Id == Cargo.Id);
39
40          if (existing is null)
41          {
42              return NotFound();
43          }
44
45          existing.Name = Cargo.Name.Trim();
46          existing.Category = Cargo.Category.Trim();
47          existing.Quantity = Cargo.Quantity;
48          existing.Location = Cargo.Location.Trim();
49          existing.Weight = Cargo.Weight;
50          existing.Description = Cargo.Description.Trim();
51          existing.IsAvailable = Cargo.IsAvailable;
52
53          TempData["SuccessMessage"] = $"Груз «{existing.Name}» успешно обновлён.";
54
55          return RedirectToPage("./Index");
56      }
57  }
```

**Рис. 9.** Pages/Cargo/Edit.cshtml.cs — сохранение изменений

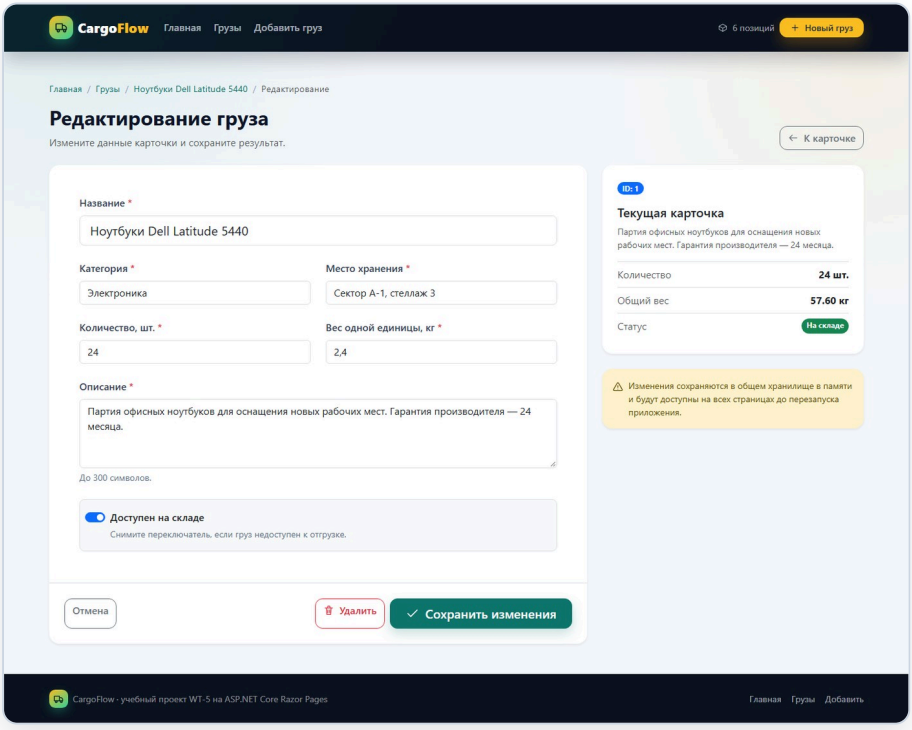



**Рис. 10.** Страница редактирования: форма предзаполнена данными груза

> **ЕДИНЫЙ СЦЕНАРИЙ**
> Форма редактирования использует ту же модель и те же правила валидации, что и добавление. Отличается только обработчик: он обновляет существующую запись, а не создаёт новую.

# Главная страница

Главная показывает состояние склада: количество позиций, доступные грузы, суммарное количество единиц и общий вес. Ниже — возможности приложения и три последних добавленных груза.

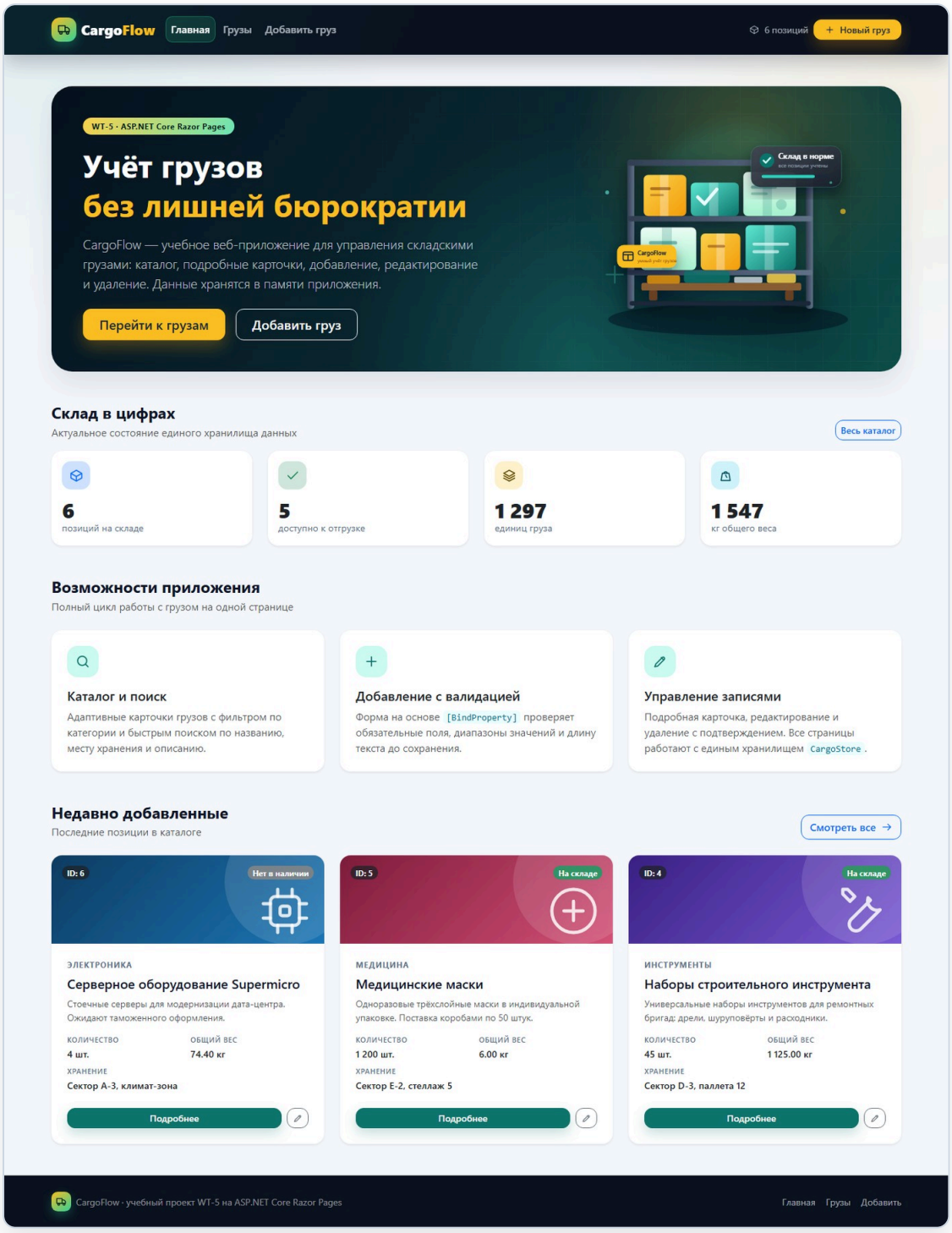



**Рис. 11.** Главная страница CargoFlow с показателями склада

**ОФОРМЛЕНИЕ**

Тёмно-зелёный hero-блок, жёлтый акцент, SVG-иллюстрация склада и карточки Bootstrap создают цельный визуальный стиль. Все иконки — инлайновый SVG-спрайт, без внешних зависимостей.

# Каталог и форма добавления

Каталог выводит грузы адаптивными карточками: на широком экране — три колонки, на планшете — две, на телефоне — одна. Над списком расположены поиск и фильтр по категории.

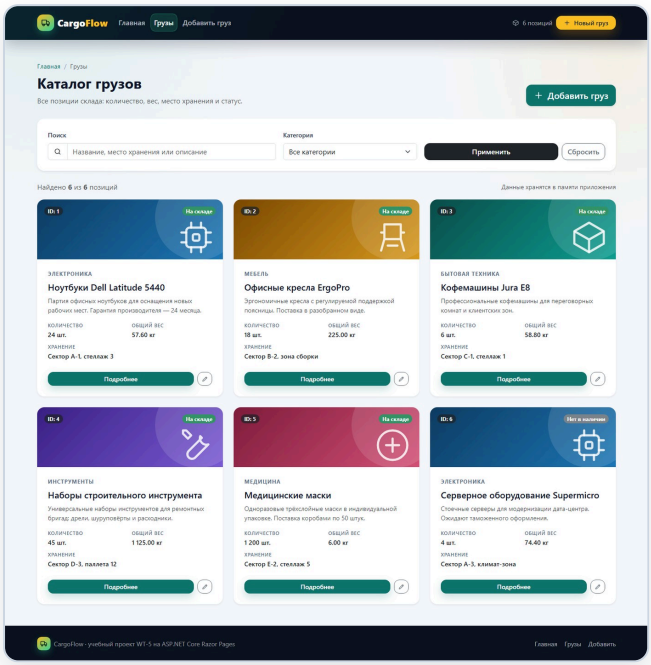



Рис. 12. Каталог с поиском и фильтром

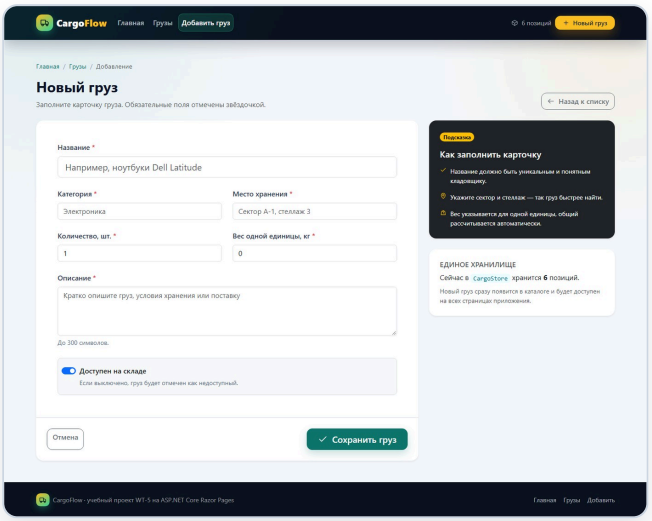



Рис. 13. Форма добавления груза

## Что видно в интерфейсе

- ✓ Поиск по названию, месту хранения и описанию, а также фильтр по категории.
- ✓ Карточка содержит статус, категорию, количество, общий вес и место хранения.
- ✓ Кнопки «Подробнее» и «Редактировать» ведут на соответствующие Razor Pages.
- ✓ Счётчик «Найдено N из M» и бейдж активного фильтра помогают ориентироваться в выборке.

> **ПОИСК ЧЕРЕЗ QUERY STRING**
>
> Параметры формы попадают в адрес страницы: `/Cargo/Index?Search=ноутбук&Category=Электроника`. Ссылкой на такую выборку можно поделиться, а обновление страницы не сбрасывает фильтр — JavaScript для поиска не требуется.

| Параметр | Поведение |
|---|---|
| `Search` | Поиск без учёта регистра по названию, месту хранения и описанию |
| `Category` | Точное совпадение категории; пустое значение означает «Все категории» |
| Сброс | Ссылка на `/Cargo/Index` без параметров очищает выборку |

# Подробности и удаление

Карточка подробностей показывает все поля груза, включая вычисленные общий вес и статус. Удаление требует явного подтверждения и показывает, какая именно запись будет удалена.

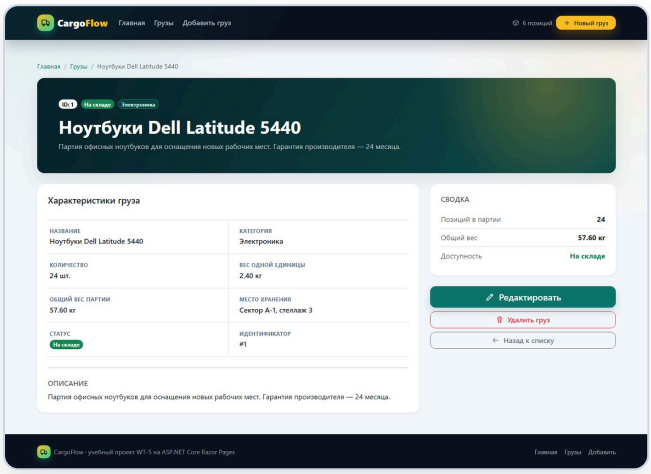



Рис. 14. Подробная карточка груза

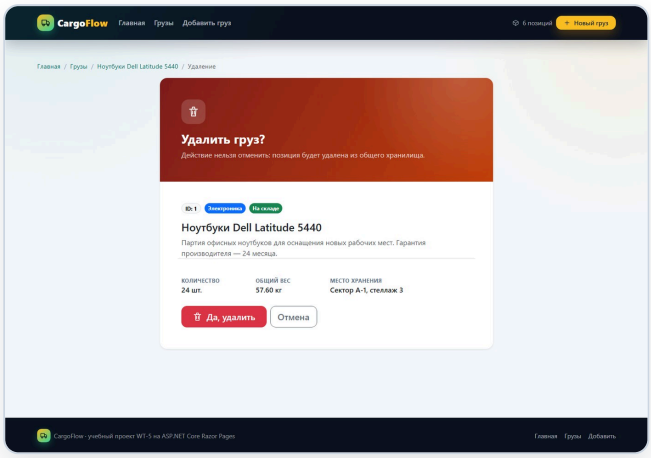



Рис. 15. Подтверждение удаления

## Сценарии

- ✓ «Подробнее» открывает `/Cargo/Details?id=…` и показывает полную информацию о грузе.
- ✓ «Редактировать» открывает форму с уже заполненными полями.
- ✓ «Удалить» открывает страницу подтверждения; фактическое удаление выполняется POST-запросом.
- ✓ После редактирования или удаления пользователь возвращается в каталог и видит уведомление.

## Форма подтверждения

```
<form method="post">
    <input type="hidden" asp-for="Cargo.Id" />
    <button type="submit" class="btn btn-danger">Удалить</button>
    <a asp-page="./Index" class="btn btn-outline-secondary">Отмена</a>
</form>
```

> **ЗАЩИТА ОТ СЛУЧАЙНОГО УДАЛЕНИЯ**
> Кнопка удаления находится только на странице подтверждения, а сам обработчик `OnPost(int id)` повторно проверяет существование записи и возвращает `NotFound()` для неизвестного идентификатора.

# Результат добавления

После успешного POST-запроса выполняется редирект в каталог. Сообщение хранится в `TempData`, поэтому отображается один раз и не «залипает» при обновлении страницы.

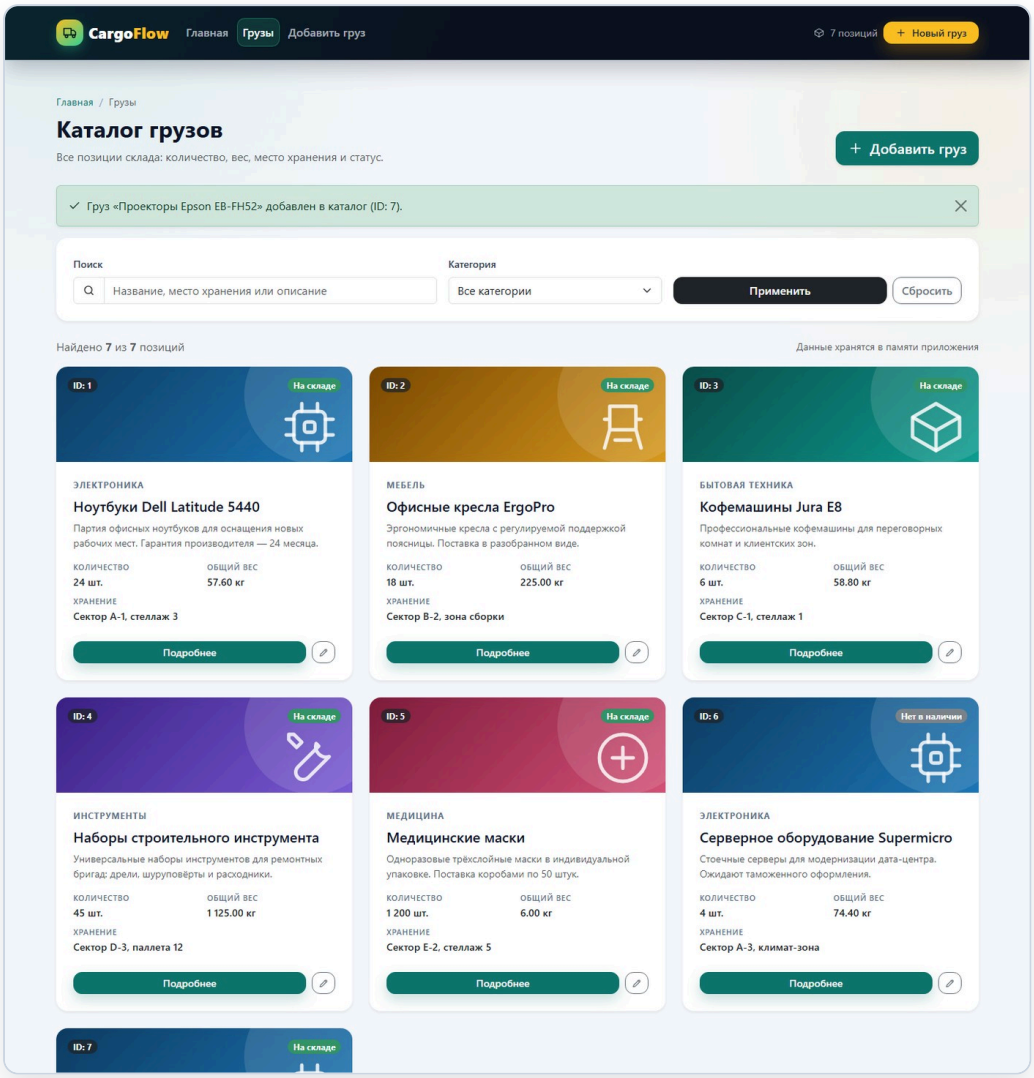



**Рис. 16.** Каталог после добавления: зелёное уведомление и новая запись

```
TempData["SuccessMessage"] =
    $"Груз «{Cargo.Name}» добавлен в каталог (ID: {Cargo.Id}).";

return RedirectToPage("./Index");
```

**POST/REDIRECT/GET**

Пользователь не остаётся на URL обработчика POST: обновление страницы не создаёт дубликат груза, а сообщение об успехе показывается ровно один раз.

# Мобильная версия

Интерфейс проверен в эмуляции экрана шириной 390 px. Сетка Bootstrap перестраивается в одну колонку, навигация сворачивается в «бургер», а карточки и формы остаются читаемыми.

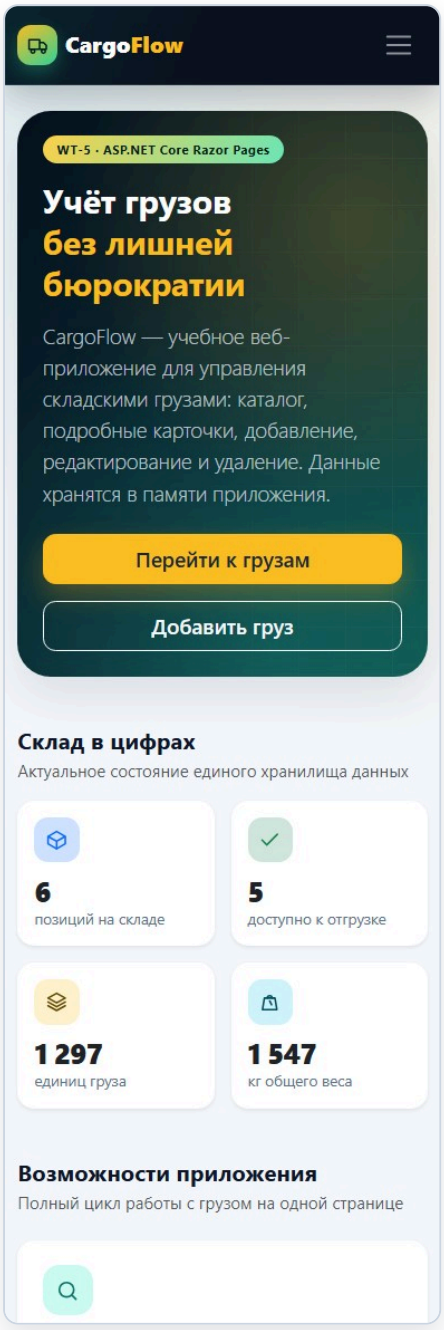



Рис. 17. Главная на телефоне

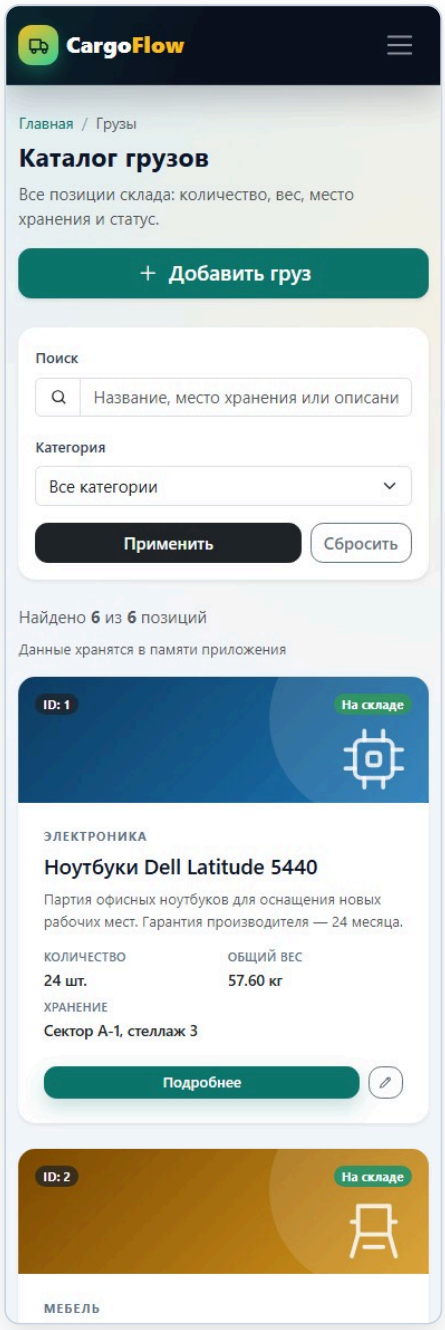



Рис. 18. Каталог на телефоне

**BOOTSTRAP-СЕТКА**

Классы `col-12 col-md-6 col-lg-4` задают одну, две и три колонки соответственно. Навигация использует `navbar-expand-lg` и `collapse`, поэтому на узком экране меню не занимает всю шапку.

# Единый layout и Bootstrap 5

`_Layout.cshtml` задаёт общую шапку, навигацию, контейнер контента и подвал. Все страницы подключаются через `_ViewStart.cshtml` и получают одинаковое оформление.

Pages/Shared/_Layout.cshtml — Единый layout и Bootstrap Navbar

```cshtml
<header>
    <nav class="navbar navbar-expand-lg navbar-dark app-navbar sticky-top">
        <div class="container">
            <a class="navbar-brand d-flex align-items-center gap-2" asp-page="/Index">
                <span class="brand-mark">
                    <svg class="icon" aria-hidden="true"><use href="#i-truck" /></svg>
                </span>
                <span class="brand-text">Cargo<span class="text-accent">Flow</span></span>
            </a>
            <button class="navbar-toggler border-0" type="button" data-bs-toggle="collapse"
                    data-bs-target="#mainNavigation" aria-controls="mainNavigation"
                    aria-expanded="false" aria-label="Открыть навигацию">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="mainNavigation">
                <ul class="navbar-nav me-auto mb-2 mb-lg-0 gap-lg-1">
                    <li class="nav-item">
                        <a class="@NavClass("/Index")" asp-page="/Index">Главная</a>
                    </li>
                    <li class="nav-item">
                        <a class="@NavClass("/Cargo/Index")" asp-page="/Cargo/Index">Грузы</a>
                    </li>
                    <li class="nav-item">
                        <a class="@NavClass("/Cargo/Create")" asp-page="/Cargo/Create">Добавить груз</a>
                    </li>
                </ul>
                <div class="d-flex flex-column flex-lg-row align-items-lg-center gap-2">
                    <span class="navbar-text d-none d-xl-inline small">
                        <svg class="icon me-1" aria-hidden="true"><use href="#i-box" /></svg>
                        @CargoStore.Cargoes.Count позиций
                    </span>
                    <a asp-page="/Cargo/Create" class="btn btn-sm btn-accent px-3">
                        <svg class="icon me-1" aria-hidden="true"><use href="#i-plus" /></svg>
                        Новый груз
                    </a>
                </div>
            </div>
        </div>
    </nav>
```

Рис. 19. `Pages/Shared/_Layout.cshtml` — Bootstrap Navbar и активный пункт меню

## Компоненты интерфейса

- ✓ `sticky-top` удерживает навигацию сверху при прокрутке.
- ✓ `NavClass()` подсвечивает активный пункт меню по текущему пути.
- ✓ SVG-спрайт в layout хранит иконки и используется через `<use href="#i-..." />`.
- ✓ Bootstrap CSS/JS лежат локально в `wwwroot/lib` — приложение не зависит от CDN.
- ✓ Подвал содержит ссылки на основные страницы и GitHub-репозиторий.

> **ЕДИНЫЙ СТИЛЬ**
>
> Тема вынесена в `wwwroot/css/site.css`: фирменные цвета, тени, hover-эффекты карточек и адаптивные правила. Страницы не дублируют стили.

# Dockerfile и запуск контейнера

Используется многоступенчатая сборка: на этапе `build` восстанавливаются зависимости и выполняется `dotnet publish`, а в финальный образ копируется только опубликованное приложение.

**Dockerfile** — Многоступенчатая сборка Docker-образа

```dockerfile
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src

COPY wt-lab5-5razor-yushkevich.csproj ./
RUN dotnet restore

COPY . .
RUN dotnet publish -c Release -o /app/publish /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS final
WORKDIR /app

COPY --from=build /app/publish .

ENV ASPNETCORE_URLS=http://0.0.0.0:10000
ENV PORT=10000
EXPOSE 10000

ENTRYPOINT ["dotnet", "wt-lab5-5razor-yushkevich.dll"]
```

**Рис. 20.** `Dockerfile` — сборка и запуск на .NET 9

## Команды

```
docker build -t wt-lab5-5razor-yushkevich .
docker run --rm -p 8080:8080 -e PORT=8080 wt-lab5-5razor-yushkevich
```

PowerShell — Docker run и проверка контейнера

```
PS D:\wt-lab5-5razor-yushkevich> docker run -d --name wtlab5 -e PORT=8080 -p 8085:8080 wt-lab5-5razor-yushkevich:latest
9baeb90d9dc101e6f71457df878f3997fb4bc983cdfa726fb915a19b2793d651

PS D:\wt-lab5-5razor-yushkevich> docker ps --filter name=wtlab5 --format '{{.Status}} {{.Ports}}'
Up 9 minutes 0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp

PS D:\wt-lab5-5razor-yushkevich> @('http://localhost:8085/', '/Cargo/Index', '/Cargo/Create', '/Cargo/Details?id=1') | % { Invoke-WebRequest
$_ }
200 http://localhost:8085/                    len=29498
200 http://localhost:8085/Cargo/Index          len=34033
200 http://localhost:8085/Cargo/Create         len=26340
200 http://localhost:8085/Cargo/Details?id=1   len=21049

PS D:\wt-lab5-5razor-yushkevich> docker stop wtlab5
wtlab5
```

✓ Контейнер запущен, переменная PORT=8080 применена, все страницы отвечают HTTP 200

**Рис. 21.** Контейнер запущен, приложение слушает порт 8080

# Сборка Docker-образа

Образ собран локально командой `docker build`. В логе видны все стадии: восстановление пакетов, сборка проекта, публикация и формирование финального слоя.

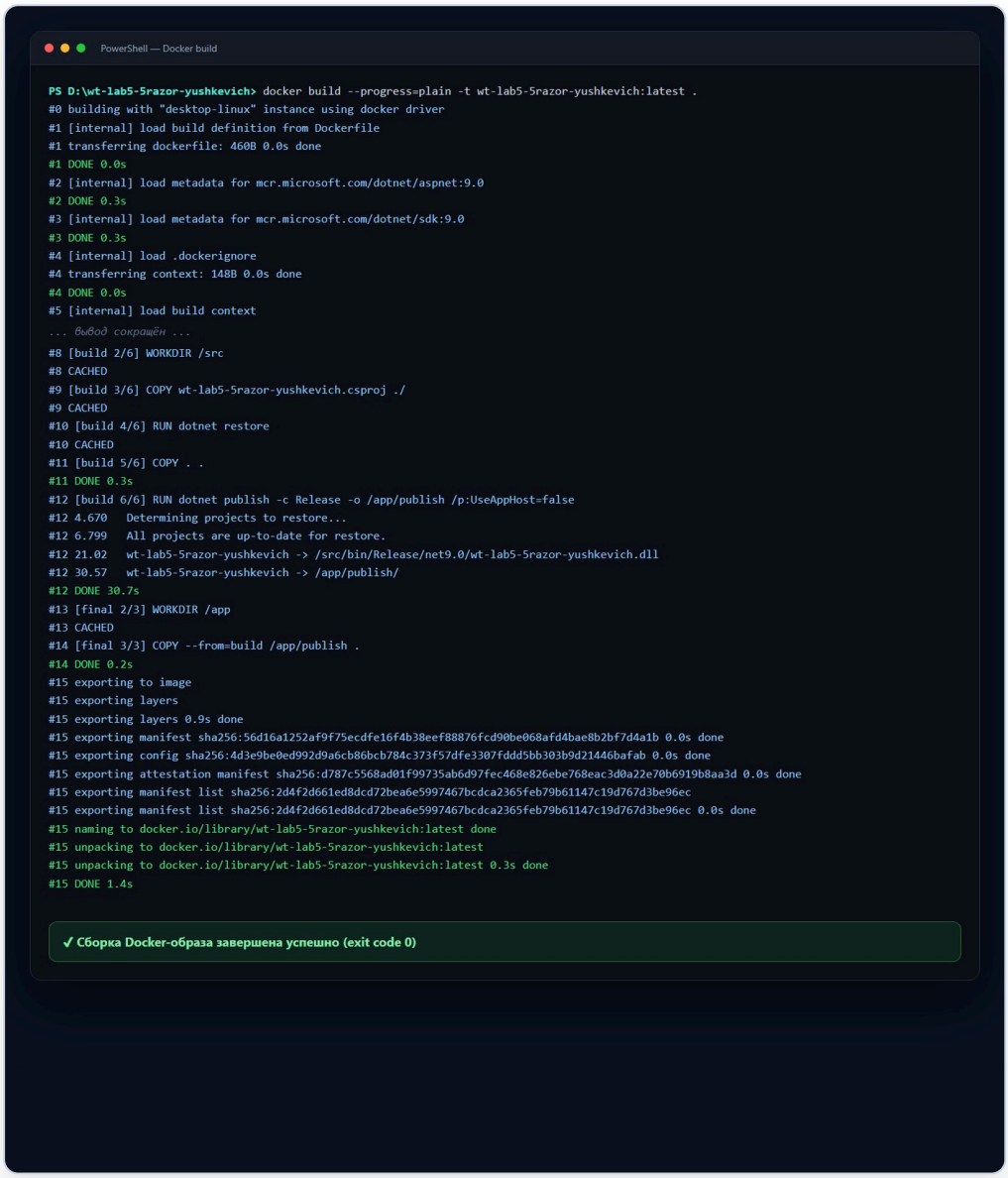



Рис. 22. Успешная сборка образа `wt-lab5-5razor-yushkevich`

## Проверка контейнера

- ✓ Контейнер запущен с `-e PORT=8080` и проброшенным портом `8080:8080`.
- ✓ Приложение читает `PORT` и слушает `http://0.0.0.0:8080`.
- ✓ Маршруты `/`, `/Cargo/Index`, `/Cargo/Create` и `/Cargo/Details?id=1` отвечают **HTTP 200**.
- ✓ Тот же Dockerfile используется Render при автоматической сборке сервиса.

> **ПОЧЕМУ PORT**
>
> Render передаёт внешний порт через переменную окружения. Приложение не должно жёстко слушать 5000/5001 — иначе платформа не сможет направить трафик в контейнер.

# Публикация на Render

Сервис создан вручную в Render Dashboard как **Web Service** с Runtime **Docker**. Источник — репозиторий GitHub, ветка `main`, Dockerfile `./Dockerfile`. При push в `main` Render автоматически пересобирает и публикует новую версию.

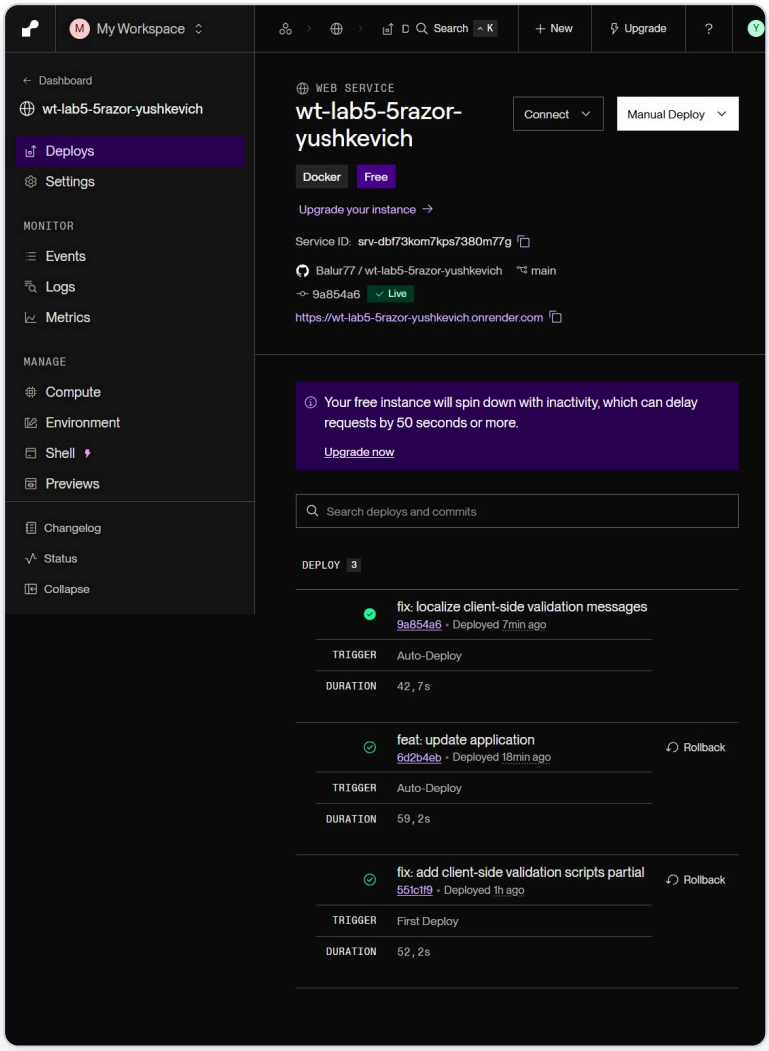



**Рис. 23.** История деплоев: три успешные сборки, последняя — 9a854a6

## Параметры сервиса

| | |
|---|---|
| Платформа | Render Web Service |
| Runtime | Docker |
| Репозиторий | Balur77 / wt-lab5-5razor-yushkevich |
| Ветка | `main` |
| Dockerfile | `./Dockerfile` |
| Статус | **Live** |
| Публичный URL | wt-lab5-5razor-yushkevich.onrender.com |

# Deployment и опубликованное приложение

Последний deployment завершился статусом **Deploy succeeded · Live**. Публичная версия открывается по адресу Render и обслуживает те же Razor Pages, что и локальный запуск.

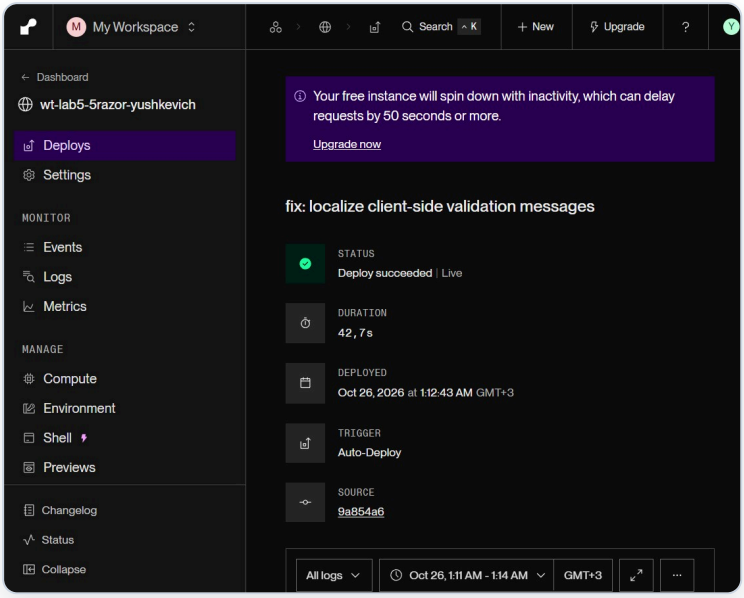



**Рис. 24.** Детали деплоя 9a854a6: статус Live, длительность 42,7 с

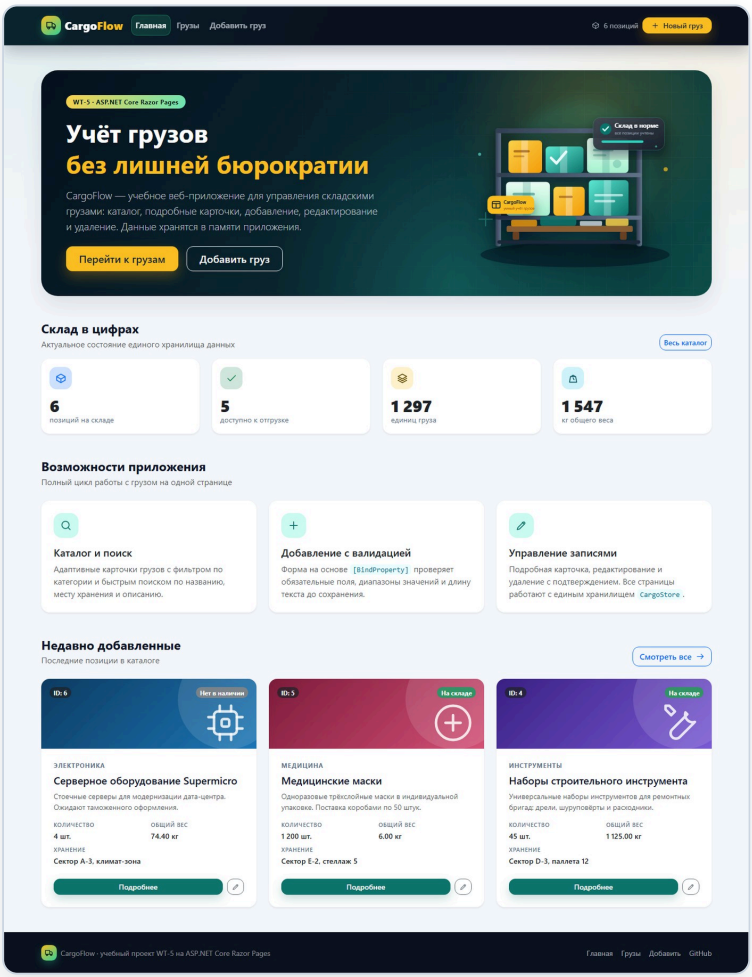



**Рис. 25.** Опубликованная главная страница на Render

# Проверка опубликованных страниц

**ПУБЛИЧНЫЙ URL**

https://wt-lab5-5razor-yushkevich.onrender.com

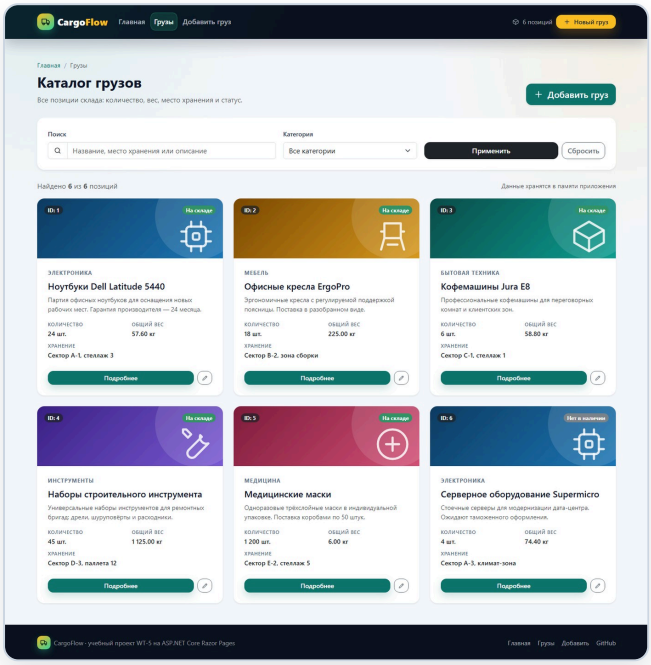



Рис. 26. Каталог на Render

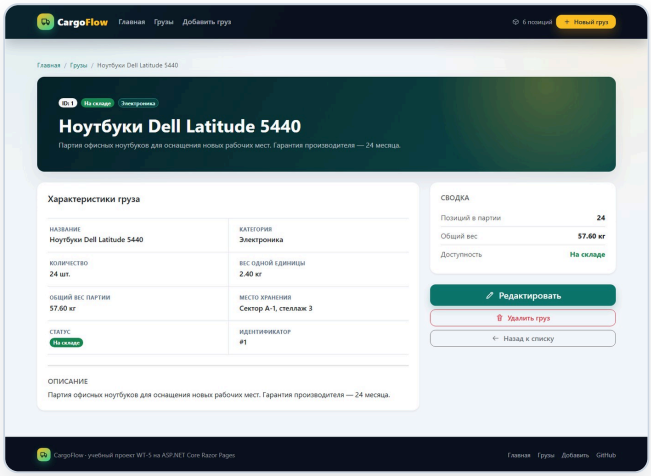



Рис. 27. Подробности груза на Render

## Что проверено на опубликованном сайте

- ✓ Главная, каталог, форма добавления и подробности отвечают **HTTP 200**.
- ✓ Статические файлы Bootstrap и стили загружаются, интерфейс совпадает с локальным.
- ✓ Клиентская валидация формы показывает сообщения на русском языке.
- ✓ Навигация и ссылки между страницами работают на публичном домене.

## Проверка маршрутов

| Маршрут | Ответ | Результат |
|---|---|---|
| `/` | **200 OK** | Главная страница загружается |
| `/Cargo/Index` | **200 OK** | Каталог и карточки грузов |
| `/Cargo/Create` | **200 OK** | Форма и клиентская валидация |
| `/Cargo/Details?id=1` | **200 OK** | Подробная карточка груза |

# Git-репозиторий и история коммитов

Исходный код размещён на GitHub. История показывает последовательную работу: базовое приложение, документация, запуск на Render, исправления валидации. Финальная версия находится в ветке `main`.

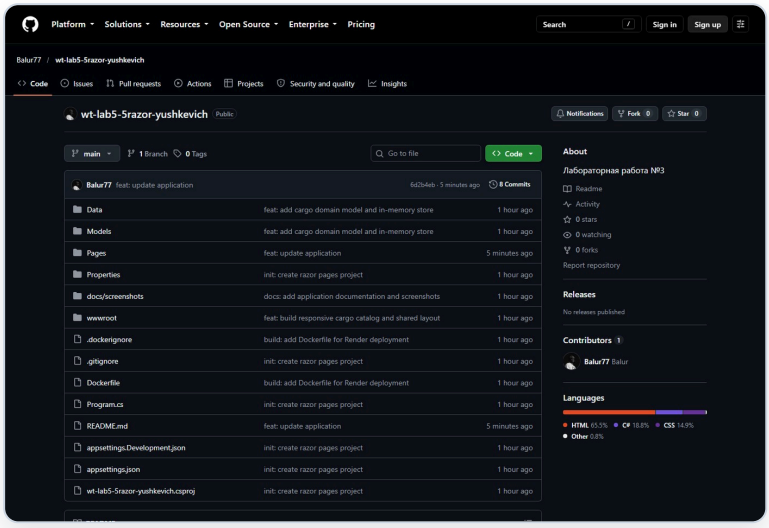



Рис. 28. Репозиторий `Balur77/wt-lab5-5razor-yushkevich` на GitHub

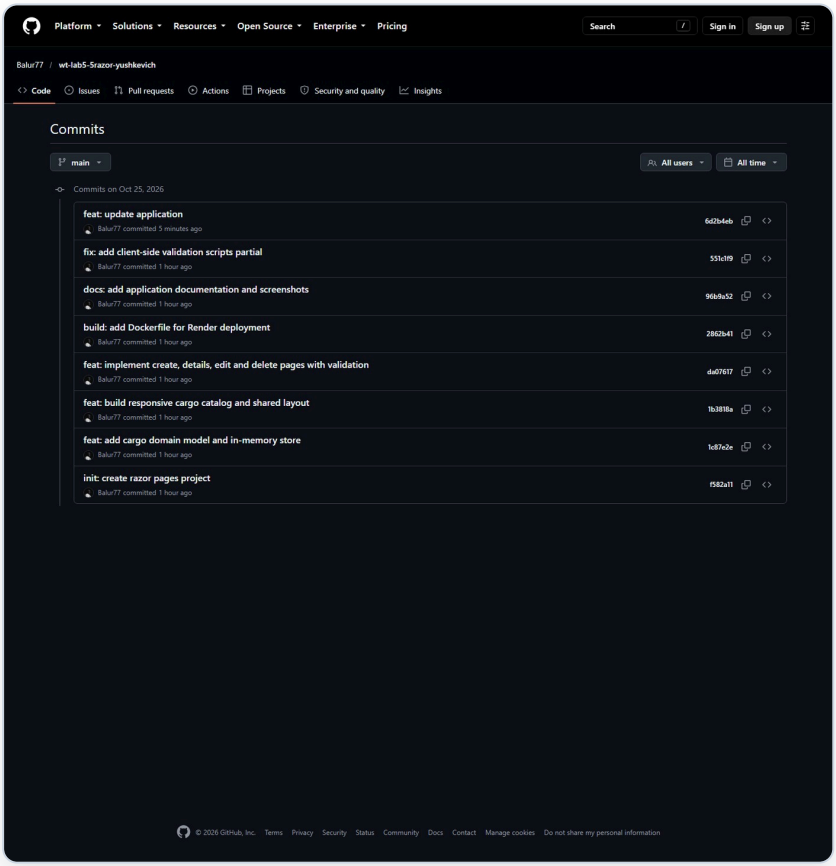



Рис. 29. История коммитов в ветке `main`

**ССЫЛКА**

https://github.com/Balur77/wt-lab5-5razor-yushkevich

# MVC и Razor Pages

Razor Pages не является отдельным фреймворком: это модель организации приложений ASP.NET Core, которая использует ту же маршрутизацию, middleware, DI, Razor и Tag Helpers. Отличается способ группировки кода страницы.

| Аспект | ASP.NET Core MVC | Razor Pages |
|---|---|---|
| Единица организации | Controller + Action + View | Страница + PageModel |
| Обработчик запроса | Метод контроллера | `OnGet()` / `OnPost()` |
| Расположение | `Controllers/` + `Views/` | `Pages/` — рядом |
| Привязка формы | Параметры action или модель | `[BindProperty]` |
| Редирект | `RedirectToAction()` | `RedirectToPage()` |
| Маршрут | Атрибуты / шаблоны контроллера | Путь файла страницы |
| Разметка | Razor View | Razor Page |
| Хорошо подходит | Крупные API и сложные сценарии | Страничные сценарии, CRUD, формы |

## Вывод по сравнению

Для приложения с преобладанием страничных сценариев Razor Pages оказался компактнее: код страницы и её обработчики находятся в одном месте, не нужно создавать отдельный контроллер и представление, а имена обработчиков напрямую отражают HTTP-метод.

MVC остаётся удобнее там, где много общих API-действий, сложная маршрутизация или несколько представлений на один контроллер. В нашем CRUD-приложении Razor Pages дал более простую структуру при сохранении всех возможностей ASP.NET Core: валидации, Tag Helpers, DI и middleware.

## Когда что выбирать

| Ситуация | Рекомендация |
|---|---|
| Страничные CRUD-сценарии и формы | Razor Pages — код страницы и обработчик рядом |
| Крупный API с множеством endpoint'ов | MVC-контроллеры или Minimal API |
| Одна страница — несколько представлений | MVC: контроллер управляет выбором View |
| Быстрый старт небольшого сайта | Razor Pages: меньше файлов и связующего кода |

# Заключение

В ходе лабораторной работы приложение учёта грузов было переведено на ASP.NET Core Razor Pages, дополнено полным CRUD-сценарием, адаптивным интерфейсом и опубликовано в Docker-контейнере на Render.

## Что освоено

- ✓ Структура Razor Pages: страница `.cshtml` и класс `PageModel`.
- ✓ Обработчики `OnGet()`, `OnGet(int id)`, `OnPost()`, `OnPost(int id)`.
- ✓ Привязка модели через `[BindProperty]` и проверка `ModelState.IsValid`.
- ✓ Tag Helpers, partial-представления, единый layout и Bootstrap 5.
- ✓ Серверная и клиентская валидация, локализация сообщений, `TempData`.
- ✓ Многоступенчатая Docker-сборка и публикация Web Service на Render.

## Итог

Приложение работает локально, в Docker-контейнере и на публичном домене Render. Все основные маршруты отвечают HTTP 200, форма добавления проверяет данные на клиенте и на сервере, а изменения сохраняются в общем хранилище в памяти. Репозиторий содержит исходный код, README, Dockerfile и скриншоты.

| | |
|---|---|
| **27**<br>страниц в отчёте | **11**<br>скриншотов кода |
| **6**<br>страниц приложения с CRUD | **3**<br>успешных deployment на Render |

**ССЫЛКИ**

Приложение: https://wt-lab5-5razor-yushkevich.onrender.com
Репозиторий: https://github.com/Balur77/wt-lab5-5razor-yushkevich

---

Отчёт подготовлен по фактически выполненной работе. Работоспособность проверена локально, в Docker и на Render.

Студент: _____ / Юшкевич Владислав / Дата: 26.10.2026